



UNIVERSITÀ DI PISA

MASTER OF SCIENCE IN COMPUTER ENGINEERING

FOUNDATIONS OF CYBERSECURITY

Digital Signature Server

Project members:

Dario BANDECCHI

Contents

1	Introduction	1
1.1	Server	1
1.2	Client	1
1.3	Utility Functions	2
2	Algorithms and Protocols	2
2.1	Cryptographic Algorithms	2
2.2	Secure Channel Establishment	2
3	Handshake Protocol	3
4	Application-layer Protocol	4
5	Format of Exchanged Messages	5
5.1	Message M1	5
5.2	Message M2	5
5.3	Standard Protocol Format	6
5.3.1	Payload of Message M3	6
5.3.2	Payload of Message M4	6
5.3.3	Payload of Message M5	7
5.4	Application-layer Messages	7
5.4.1	Request Message	7
5.4.2	Response Message	7

1 Introduction

The project implements a **digital signature server**, i.e., a server that provides a variety of services, such as generating a key pair on behalf of the user, deleting the key pair, digitally signing a specified document, and returning the public key of a specified user.

The project is implemented in C++ for Linux systems and consists of three main source files: **server.cpp**, **client.cpp** and **utils.cpp**, which contains utility functions used by both the client and the server. Communication between the client and the server is handled via **TCP** sockets (to ensure reliability and in-order delivery) over the localhost interface. All cryptographic operations, including key generation, hashing, digital signatures, and encryption, are performed using the **OpenSSL** library.

Once the TCP connection is established, both the client and the server perform their respective parts of the **handshake protocol** to achieve mutual authentication and establish a shared secret. This enables them to communicate over a secure channel that ensures perfect forward secrecy, integrity, confidentiality, and protection against replay attacks.

1.1 Server

The server is **multiprocess**, allowing multiple users to benefit from the services offered simultaneously. In particular, when a new connection request arrives, the parent process creates a new child process (using the `fork()` system call) and continues listening for further incoming connections, while the child process handles the requests from that user. Once the user closes the connection, the child process terminates.

Whenever a new client connects to the server, a secure channel must be established before handling any requests.

1.2 Client

Once connected to the server, the client prompts the user to enter their username and password. If the credentials are valid, the user can start issuing commands, namely:

- **CreateKeys**: creates and stores a pair of RSA private and public keys on behalf of the invoking user. The private key is encrypted by means of the user's password. Both keys are saved in a dedicated directory at `keys/users/username/`, where **username** corresponds to the current invoking user.
- **SignDoc** <file>: returns the digital signature of the specified document. The document is retrieved from the `documents/` folder, and the generated signature is saved in the `signatures/` folder.
- **GetPublicKey** <user>: returns the public key of the specified user. The key is saved in the `public_keys/` folder.
- **DeleteKeys**: deletes the key pair associated with the invoking user. After this operation, the user cannot generate a new key pair unless they are registered again offline. To keep track of revoked users, the current username is recorded in `data/revoked.txt`.

1.3 Utility Functions

The **utils.cpp** file implements some utility functions for communication through sockets and for composing and parsing messages of the application-layer protocol.

2 Algorithms and Protocols

2.1 Cryptographic Algorithms

As regards symmetric cryptographic algorithms, **AES-128** in **GCM** mode is used. This mode combines the AES block cipher operating in counter (CTR) mode for encryption with Galois field multiplication to generate an authentication tag, thus providing both confidentiality and authenticity.

For asymmetric cryptographic operations, such as key generation and digital signatures, **RSA** with a **3072-bit** key size is employed, ensuring an adequate security level according to current standards.

Regarding password hashing, the **PKCS5_PBKDF2_HMAC** algorithm is used to securely hash user passwords before storing them in a file. It derives a strong key from the password and a salt through multiple iterations of HMAC using **SHA-256**, protecting against brute-force and dictionary attacks. The resulting information is stored in the file **data/users.txt** in the following format:

```
username:salt:password_hash:flag_change_pwd
```

where **salt** is the unique salt used during hashing, **password_hash** is the derived key (hashed password), and **flag_change_pwd** indicates whether the user is required to change their password.

2.2 Secure Channel Establishment

To establish a secure channel between the client and the server, a slightly modified version of the **Station-to-Station protocol** is employed, as explained in detail in Section 3. In particular, the user is authenticated by means of their password, while the server authenticates itself to the user using its public key.

The server's RSA key pair is generated offline using the OpenSSL command-line interface and stored in the **keys/server/** directory.

This protocol guarantees **PFS** (Perfect Forward Secrecy) and **DA** (Direct Authentication). The derived shared secret is then used to ensure both confidentiality and integrity using AES-128 in GCM mode.

Finally, to prevent replay attacks, a **challenge-response mechanism** is used. Specifically, each message includes a newly generated random nonce in the clear, while the encrypted payload contains the nonce received in the previous message. This ensures that both parties prove their liveness and that messages cannot be reused by an attacker.

3 Handshake Protocol

Regarding the implementation of the **Handshake Protocol**, as mentioned earlier, a slightly modified version of the **Station-to-Station** protocol is employed. The sequence diagram of the protocol is shown below.

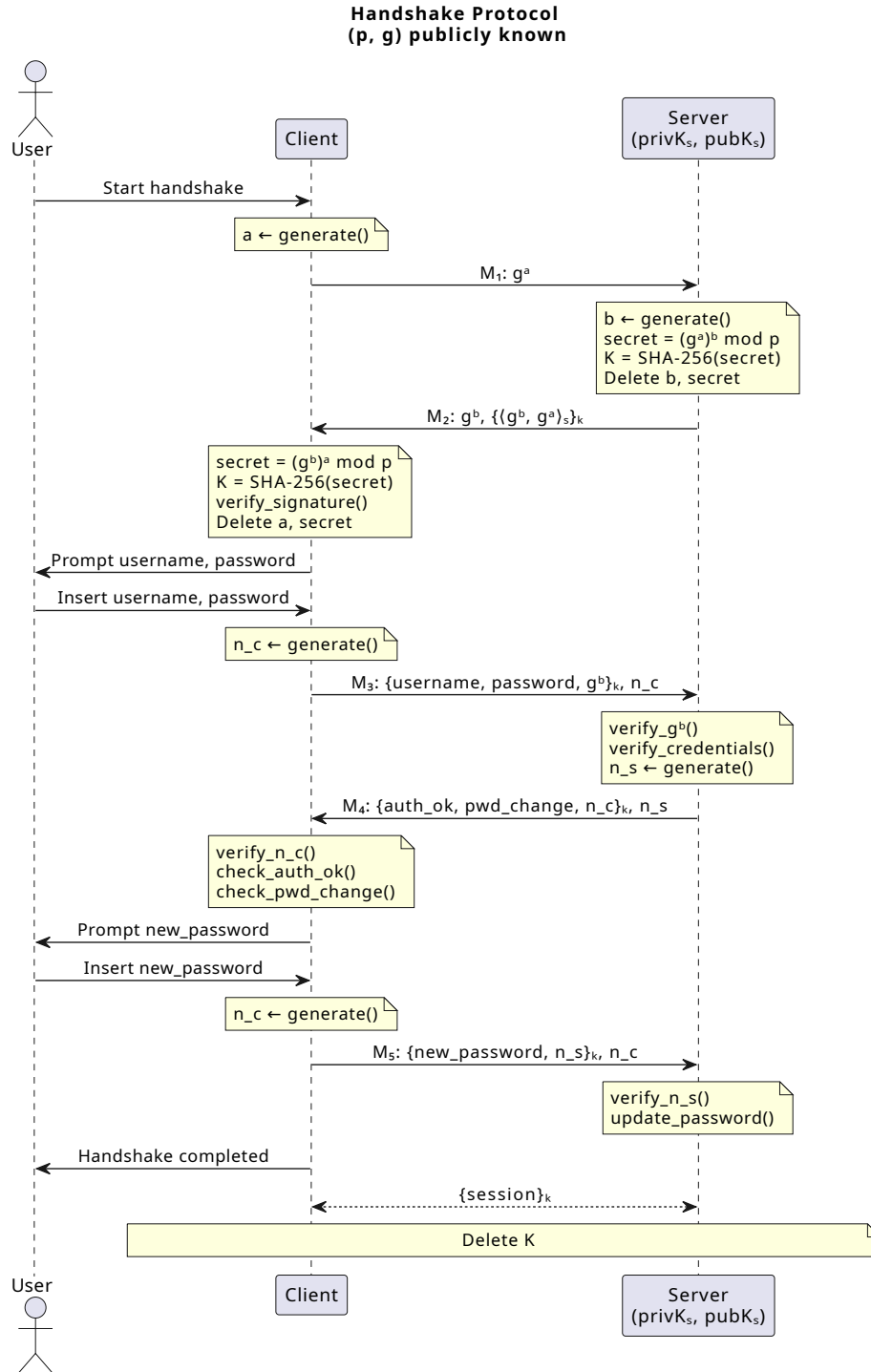


Figure 1: Handshake Protocol

Each encrypted message in the protocol also includes an **Initialization Vector** (randomly generated) and the **Authentication Tag**, along with additional length fields required for proper parsing of subsequent components. These elements are not explicitly shown in the diagram to keep it clear and concise, but are essential for secure communication. The part denoted as 'session' refers to the application-layer message exchange.

For more details on the structure of the exchanged messages, see Section 5.

4 Application-layer Protocol

The following sequence diagram illustrates the flow of messages exchanged during the session. To improve readability, and given that all messages conform to the same structure (as described in Section 5.3), only the plaintext content is shown.

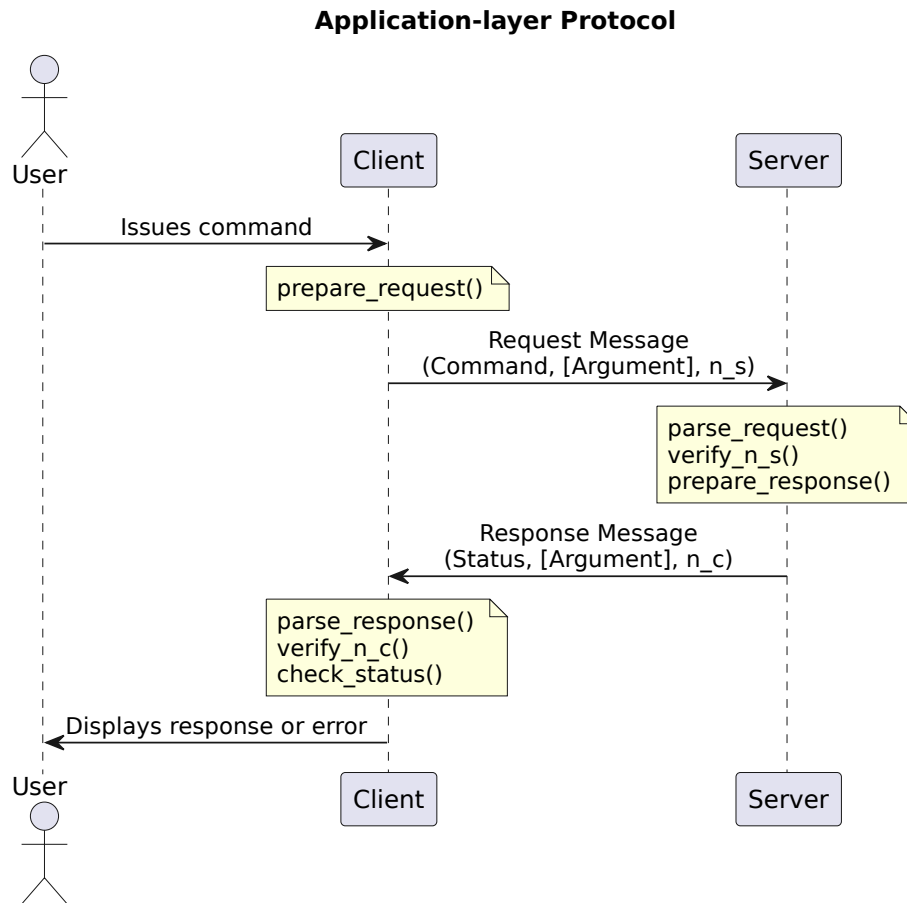


Figure 2: Application Protocol

5 Format of Exchanged Messages

This chapter describes the format of all messages exchanged during both the handshake protocol and the application-level communication.

5.1 Message M1

Message **M1** consists of the client's public value g^a , where a is a randomly generated secret exponent. This message initiates the **Diffie-Hellman** key exchange process.

Field	g^a
Size (bytes)	Variable

Table 1: Format of Message M1

5.2 Message M2

Message **M2** contains several fields important for secure communication:

- **IV (12 bytes):** Initialization vector used for encryption to ensure randomness.
- **Len g^b (4 bytes):** Length of the public key g^b .
- **g^b (variable):** The actual public key value from the server.
- **Len CT (4 bytes):** Length of the encrypted data.
- **$\{\langle g^a, g^b \rangle_S\}_K$ (variable):** The ciphertext containing the server's digital signature on both public keys, encrypted with the shared key K .
- **Tag (16 bytes):** Authentication tag to verify message integrity and authenticity.

In short, **M2** securely shares the public key g^b and an encrypted, authenticated block that proves the identity and freshness of the message.

Field	IV	Len g^b	g^b	Len CT	$\{\langle g^a, g^b \rangle_S\}_K$	Tag
Size (bytes)	12	4	Variable	4	Variable	16

Table 2: Format of Message M2

5.3 Standard Protocol Format

The remaining three handshake protocol messages, as well as all application messages, share the same structure, which is implemented by the function `compose_std_message()`. This structure is illustrated in the following.

Field	IV	Len CT	CT	Nonce	Tag
Size (bytes)	12	4	Variable	16	16

Table 3: Standard Format of Message

Each message contains an Initialization Vector of 12 bytes, a 4-byte field indicating the length of the ciphertext, the ciphertext itself with variable length, a 16-byte randomly generated nonce, and a 16-byte authentication tag. This structure ensures confidentiality and integrity of the message during communication.

Since the last three handshake protocol messages follow this structure, it suffices to illustrate only their payloads (i.e., the plaintext data).

5.3.1 Payload of Message M3

Field	Len Username	Username	Len Password	Password	g^b
Size (bytes)	4	Variable	4	Variable	Variable

Table 4: Payload of Message M3

The payload of Message **M3** consists of the length of the username (4 bytes), the username itself (variable length), the length of the password (4 bytes), the password (variable length), and the public value g^b (variable length). This payload carries the user credentials and the Diffie-Hellman server's public key which acts as a nonce.

5.3.2 Payload of Message M4

Field	auth_ok	pwd_change	n_c
Size (bytes)	1	1	16

Table 5: Payload of Message M4

The payload of Message **M4** contains two 1-byte flags: `auth_ok` indicating authentication success, and `pwd_change` signaling a password change request (in case of first user access to the service). It also includes a 16-byte nonce n_c used for freshness and replay protection.

5.3.3 Payload of Message M5

Field	Len Password	New password	n_s
Size (bytes)	4	Variable	16

Table 6: Payload of Message M5

The payload of Message **M5** contains the length of the new password (4 bytes), the new password itself (variable length), and a 16-byte nonce n_s . This message is sent only when the `pwd_change` flag in Message M4 is set to 1, indicating a password update request.

5.4 Application-layer Messages

All application messages follow the structure presented in Section 5.3, therefore, only their payloads are shown in the following.

5.4.1 Request Message

This message contains the command issued by the user, an optional argument depending on the specific command, and the last nonce received from the server.

Field	Len Command	Command	Len Argument	Argument	n_s
Size (bytes)	4	Variable	4	Variable	16

Table 7: Payload of Request Message

If no argument is present, the **Len Argument** field is set to 0, allowing the server to correctly parse the message.

5.4.2 Response Message

Similarly to the previous message, this contains the server's response to the request.

Field	Status	Len Argument	Argument	n_c
Size (bytes)	1	4	Variable	16

Table 8: Payload of Response Message

The message includes a 1-byte **Status** field, which can assume one of the following three values:

- **STATUS_LOGICAL_ERROR** (0)
- **STATUS_SUCCESS** (1)
- **STATUS_INVALID_COMMAND** (2)

Depending on the context, the message may include two additional fields: in case of a **GetPublicKey** request, these fields contain the user's public key and its length; for a **SignDoc** request, they contain the signature and its length.

As in previous messages, the payload ends with the last client's nonce, used to ensure freshness.